

MySQL Data Types and Constraints Reference Guide (MySQL 8)

Introduction

Choosing the correct data type is one of the most important database design decisions. A well-designed schema improves:

- Storage efficiency
- Query performance
- Data integrity
- Index performance
- Scalability

This guide covers the major MySQL 8 data types, their storage requirements, advantages, disadvantages, and when to use them.

1. Numeric Data Types

Numeric data types store whole numbers, decimal numbers, and floating-point values.

Data Type	Storage	Range	Best Used For	Advantages	Disadvantages
TINYINT	1 byte	-128 to 127 (Signed) 0–255 (Unsigned)	Flags, Boolean values, ratings	Very small storage	Very limited range
SMALLINT	2 bytes	±32,767	Small counters	Efficient	Limited values
MEDIUMINT	3 bytes	±8.3 million	Inventory quantities	More range than SMALLINT	Less commonly used
INT (INTEGER)	4 bytes	±2.1 billion	IDs, quantities	Fast and common	Uses more space than smaller integers
BIGINT	8 bytes	±9 quintillion	Large IDs, financial systems	Huge capacity	Larger indexes and storage

UNSIGNED

Unsigned numbers only store positive values.

Example

```
age TINYINT UNSIGNED
```

Range becomes:

```
0 - 255
```

instead of

```
-128 - 127
```

Advantages

- Doubles positive range
- Saves using a larger datatype

Disadvantages

- Cannot store negative values

Exact Decimal Types

Data Type	Storage	Best Used For	Advantages	Disadvantages
DECIMAL(p,s)	5-17 bytes	Money	Exact precision	Slightly slower than FLOAT
NUMERIC	Same as DECIMAL	Financial data	ANSI standard	Same as DECIMAL

Example

```
salary DECIMAL(10,2)
```

Stores

```
99999999.99
```

Advantages

- Exact calculations
- No rounding errors

Disadvantages

- Slightly slower than floating point

Floating Point Types

Data Type	Storage	Precision	Best Used For
FLOAT	4 bytes	Approximate	Scientific measurements
DOUBLE	8 bytes	More accurate	GPS coordinates, engineering

Advantages

- Very fast
- Small storage

Disadvantages

- Precision errors
- Never use for money

Example

```
temperature FLOAT
```

2. String Data Types

CHAR

Fixed-length text.

```
country_code CHAR(2)
```

Storage

Length specified

Example

```
US  
ZA  
KE
```

Advantages

- Very fast
- Excellent for fixed-length codes

Disadvantages

- Wastes space for shorter values

VARCHAR

Variable-length text.

```
name VARCHAR(255)
```

Storage

Actual characters + 1-2 bytes

Advantages

- Saves storage
- Most commonly used text type

Disadvantages

- Slightly slower than CHAR

TEXT Types

Type	Maximum Size	Storage
TINYTEXT	255 bytes	1-byte length
TEXT	65 KB	2-byte length
MEDIUMTEXT	16 MB	3-byte length
LONGTEXT	4 GB	4-byte length

Advantages

- Store large text

Disadvantages

- Cannot have full indexes without specifying a prefix
- Slower than VARCHAR

Best for

- Blog posts
- Product descriptions
- User comments

ENUM

Stores one value from a predefined list.

Example

```
status ENUM(  
  'Pending',  
  'Completed',  
  'Cancelled'  
)
```

Advantages

- Saves storage
- Prevents invalid values

Disadvantages

- Difficult to modify later
- Not portable across database systems

Storage

1-2 bytes

SET

Stores multiple values.

Example

```
permissions SET(  
  'Read',  
  'Write',  
  'Delete'  
)
```

Advantages

- Compact storage

Disadvantages

- Difficult to query
- Generally not recommended for normalized databases

3. Binary Data Types

Data Type	Maximum Size	Usage
BINARY	Fixed binary data	Encryption
VARBINARY	Variable binary	Password hashes
BLOB	64 KB	Images
MEDIUMBLOB	16 MB	Audio

Data Type	Maximum Size	Usage
LONGBLOB	4 GB	Videos

Advantages

- Store binary files

Disadvantages

- Large backups
- Slower queries

4. Date and Time Types

Data Type	Storage	Range	Best Used For
DATE	3 bytes	1000–9999	Birthdays
TIME	3 bytes	±838 hours	Durations
YEAR	1 byte	1901–2155	Manufacturing year
DATETIME	5 bytes (8 bytes with fractional seconds)	1000–9999	Historical timestamps
TIMESTAMP	4 bytes	1970–2038	Audit columns

Example

```
created_at DATETIME
```

Advantages

- Efficient date calculations
- Built-in formatting functions

Disadvantages

- TIMESTAMP has a limited range (the Year 2038 limit)

5. JSON

Stores structured JSON documents.

Example

payload JSON

Example value

```
{
  "cpu_pct": 35.2,
  "battery": 82
}
```

Storage

Variable

Advantages

- Flexible schema
- Supports nested objects and arrays
- Native JSON functions

Disadvantages

- Larger storage than normalized tables
- Slower if heavily queried without generated columns and indexes

6. Spatial Data Types (GIS)

Type	Usage
POINT	GPS location
LINESTRING	Roads
POLYGON	Geographic boundaries
MULTIPOINT	Multiple locations
MULTIPOLYGON	Maps

Advantages

- Built-in GIS support
- Spatial indexing

Disadvantages

- Specialized use cases

7. Boolean

MySQL treats

BOOLEAN

as

TINYINT(1)

Storage

1 byte

Example

is_active BOOLEAN DEFAULT TRUE

Storage Summary

Type	Storage
TINYINT	1 byte
SMALLINT	2 bytes
MEDIUMINT	3 bytes
INT	4 bytes
BIGINT	8 bytes
FLOAT	4 bytes
DOUBLE	8 bytes

Type	Storage
DATE	3 bytes
TIME	3 bytes
YEAR	1 byte
DATETIME	5–8 bytes
TIMESTAMP	4–7 bytes
CHAR	Fixed
VARCHAR	Variable
TEXT	Up to 4 GB
JSON	Variable

MySQL Constraints

Constraints enforce rules on data, ensuring accuracy, consistency, and integrity.

1. PRIMARY KEY

Uniquely identifies each row.

Example

```
user_id INT PRIMARY KEY
```

Advantages

- Prevents duplicates
- Automatically indexed
- One per table

Use cases

- Customer IDs
 - Order IDs
 - Product IDs
-

2. FOREIGN KEY

Creates a relationship between tables.

Example

```
FOREIGN KEY(user_id)  
REFERENCES users(user_id)
```

Advantages

- Enforces referential integrity
- Prevents orphan records

Disadvantages

- Slight overhead on inserts, updates, and deletes

Use cases

- Orders → Users
 - Order Items → Products
-

3. UNIQUE

Prevents duplicate values.

Example

```
email VARCHAR(255) UNIQUE
```

Advantages

- Allows fast lookups
- Ensures uniqueness

Use cases

- Email addresses
 - Employee numbers
 - Product SKUs
-

4. NOT NULL

Prevents missing values.

Example

```
first_name VARCHAR(50) NOT NULL
```

Advantages

- Improves data quality
- Simplifies queries

Use cases

- Required fields such as names and dates
-

5. CHECK

Restricts values according to a condition.

Example

```
salary DECIMAL(10,2),  
CHECK (salary >= 0)
```

Advantages

- Enforces business rules in the database

Disadvantages

- Older MySQL versions ignored CHECK constraints; MySQL 8 enforces them.

Use cases

- Positive prices
 - Valid ages
 - Percentage ranges
-

6. DEFAULT

Automatically supplies a value if none is provided.

Example

```
created_at DATETIME  
DEFAULT CURRENT_TIMESTAMP
```

Advantages

- Reduces NULL values
- Simplifies INSERT statements

Use cases

- Creation timestamps
 - Active flags
 - Default statuses
-

7. AUTO_INCREMENT

Automatically generates sequential numeric values.

Example

```
order_id INT AUTO_INCREMENT
```

Advantages

- No need to manually generate IDs
- Prevents duplicate identifiers

Disadvantages

- Gaps can occur if transactions roll back or rows are deleted.
-

Constraint Summary

Constraint	Purpose	Typical Usage
PRIMARY KEY	Unique row identifier	IDs
FOREIGN KEY	Relate tables	Database relationships
UNIQUE	Prevent duplicates	Email, SKU
NOT NULL	Mandatory value	Names, prices
CHECK	Validate data	Positive numbers, age limits
DEFAULT	Automatic value	Status, timestamps
AUTO_INCREMENT	Generate IDs	Primary keys

Best Practices

1. Choose the smallest suitable data type to reduce storage and improve cache efficiency.
2. Use `INT` for most identifiers; use `BIGINT` only when very large ranges are required.
3. Use `DECIMAL` for financial data and avoid `FLOAT` or `DOUBLE` for currency.
4. Prefer `VARCHAR` over `CHAR` unless the stored values have a fixed length.
5. Use `BOOLEAN` for true/false values to improve readability.
6. Add `NOT NULL` wherever a value is mandatory.
7. Use `UNIQUE` for natural keys such as email addresses or product SKUs.
8. Use `FOREIGN KEY` constraints to maintain referential integrity.
9. Index columns that are frequently searched, joined, or sorted.
10. Use `JSON` only for semi-structured data that does not fit well into a normalized schema; for frequently queried JSON attributes, use generated columns and indexes.